Minimization and K-maps

Original slides by: Jarrett Billingsley
Modified with bits from: Bruce Childers, David Wilkinson

#E

CS 0447
Introduction to
Computer Programming

Luís Oliveira

Fall 2020

Class announcements

- Check the schedule
  - I'll post examples of circuits and suggestions of exercises
- Labs will be submitted via Gradescope

How ICs are made in 3 slides

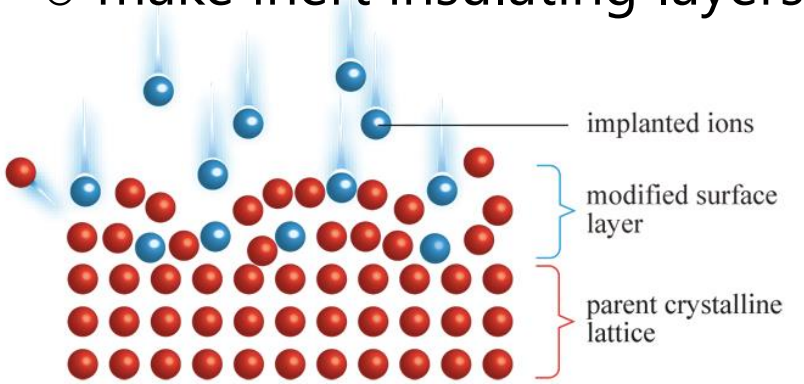
(another For Fun™ section)

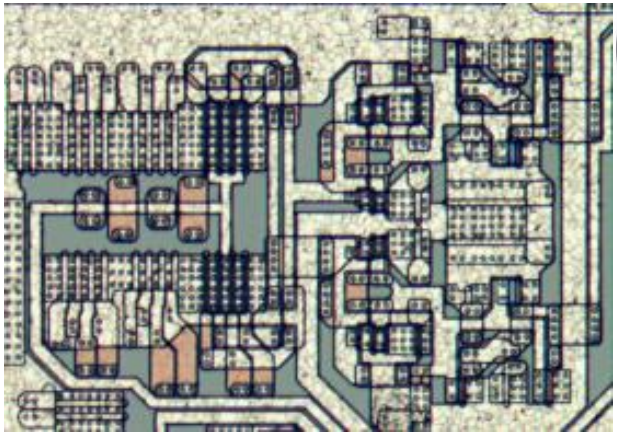
How ICs (integrated circuits) are made

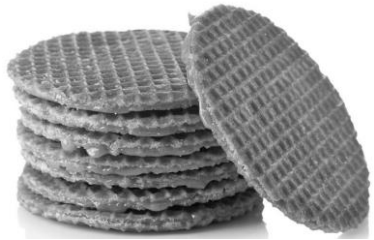
- silicon is purified and grown into a **monolithic crystal** *(extremely expensive)*
- this is sliced thinly to make **wafers**
  - gooey caramel is put between them to make **stroopwafel**
- a series of complicated photochemical processes do things like:
  - change its electrical properties
  - make wires to connect things
  - make inert insulating layers





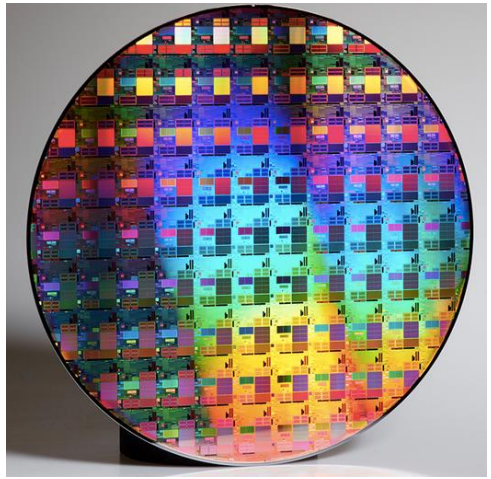


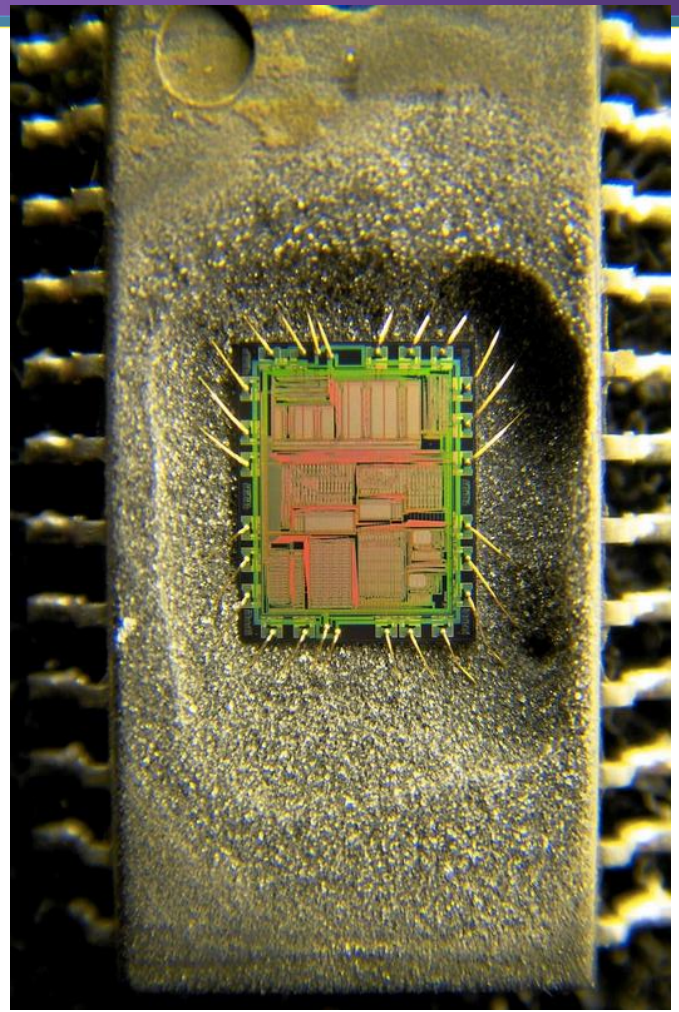




How ICs (integrated circuits) are made

- many ICs are printed on one wafer
- the wafers are **diced** (chopped up)
- the ICs are **tested**
- the ICs are **mounted** in a package ->
- they're tested *again*
- *then* they're ready to sell





Manufacturing yield

- ICs are **tiny and complex**
- silicon crystals can have **defects**
- a tiny speck of dust during production can **ruin an entire chip**
- the **yield** is the **percentage of usable chips**
  - **bigger** chips have **smaller** yields: more opportunities for mistakes!
- the **size of the silicon** is **the biggest factor** in the price of an IC
  - huge ICs (several *cm* on each side!), such as very high-resolution camera sensors, can cost **tens of thousands of dollars!**
- manufacturers can also **bin** resulting chips
  - bug-free ones can be sold as Core i7s for *lotsa money*
  - slightly malformed ones can be sold as Core i5s and i3s
  - the ones they sweep off the floor are the Celerons

Logic Minimization

Silicon is expensive, rocks are slow

- **Logic minimization** means using the **smallest number** of gates/transistors possible to implement a boolean function
  - a **boolean function** is anything we've talked about
  - it has boolean inputs, and boolean outputs
    - **Less inputs** can also **improve speed**
- Fewer transistors means:
  - smaller area:
    - **cheaper chips!**
    - **more stuff** on one chip!
    - **smaller chance** of manufacturing **defects!**
  - less gate delay:
    - **faster circuits!**

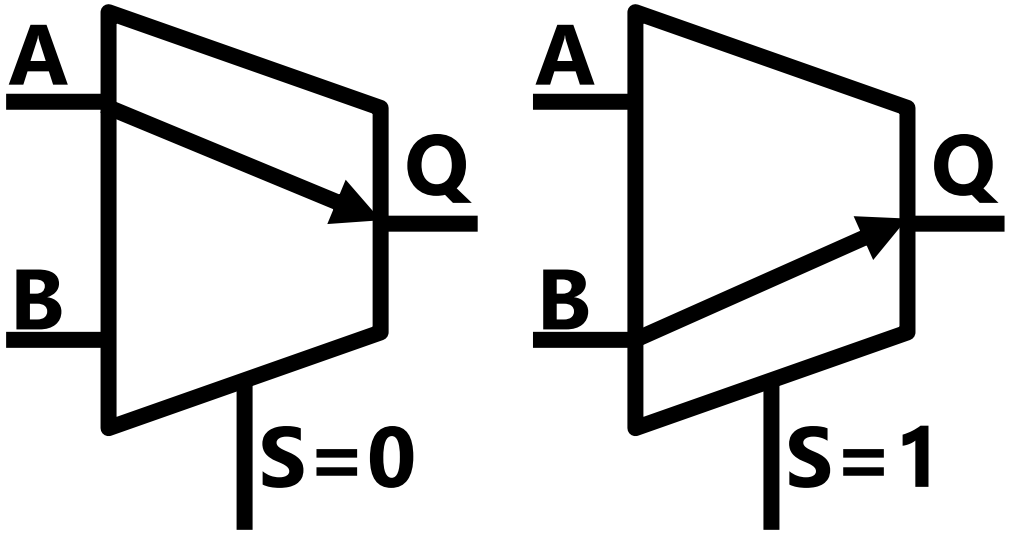
Minimizing Booleans

- How do we minimize Boolean functions?
  - Logical adjacency!
- If in two Boolean terms being ORed only one of the variables changes then it can be removed
  - $\boldsymbol{Q = A\overline{B} + AB = A(\overline{B} + B) = A}$
  - $\boldsymbol{Q = A\overline{B}C + A\overline{B}\overline{C} = A\overline{B}(C + \overline{C}) = A\overline{B}}$

- To minimize a Boolean expression
  - Find the terms where **only one variable changes**
  - Eliminate that variable

A first try

- The truth table for a **two-input 1-bit multiplexer.**



S	Q
0	A
0	A
0	A
0	A
1	B
1	B
1	B
1	B

S	A	B	Q
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

A first try

- The truth table for a **two-input 1-bit multiplexer.**

	S	A	B	Q
	0	0	0	0
	0	0	1	0
$\bar{S}A\bar{B}$	0	1	0	1
$\bar{S}AB$	0	1	1	1
	1	0	0	0
$S\bar{A}B$	1	0	1	1
	1	1	0	0
SAB	1	1	1	1

$$Q = \bar{S}A\bar{B} + \bar{S}AB + S\bar{A}B + SAB$$

Are there any adjacencies?

Well… I guess we don't need those variables

$$Q = \bar{S}A + \cancel{\bar{S}A} + SB + \cancel{SB}$$

This seems easy…

$$Q = \bar{S}A + SB$$

It makes sense, right?

- We know that **if S=0** then the **output is A**
- We know that **if S=1** then the **output is B**
  - We **don't care** about one of the variables
- So, let's solve this truth table
  1. find all rows with an **output of 1**
  2. for each one, write an AND of all the inputs, with NOTs on the 0s
  3. **eliminate duplicate terms**
  4. OR the remaining terms together

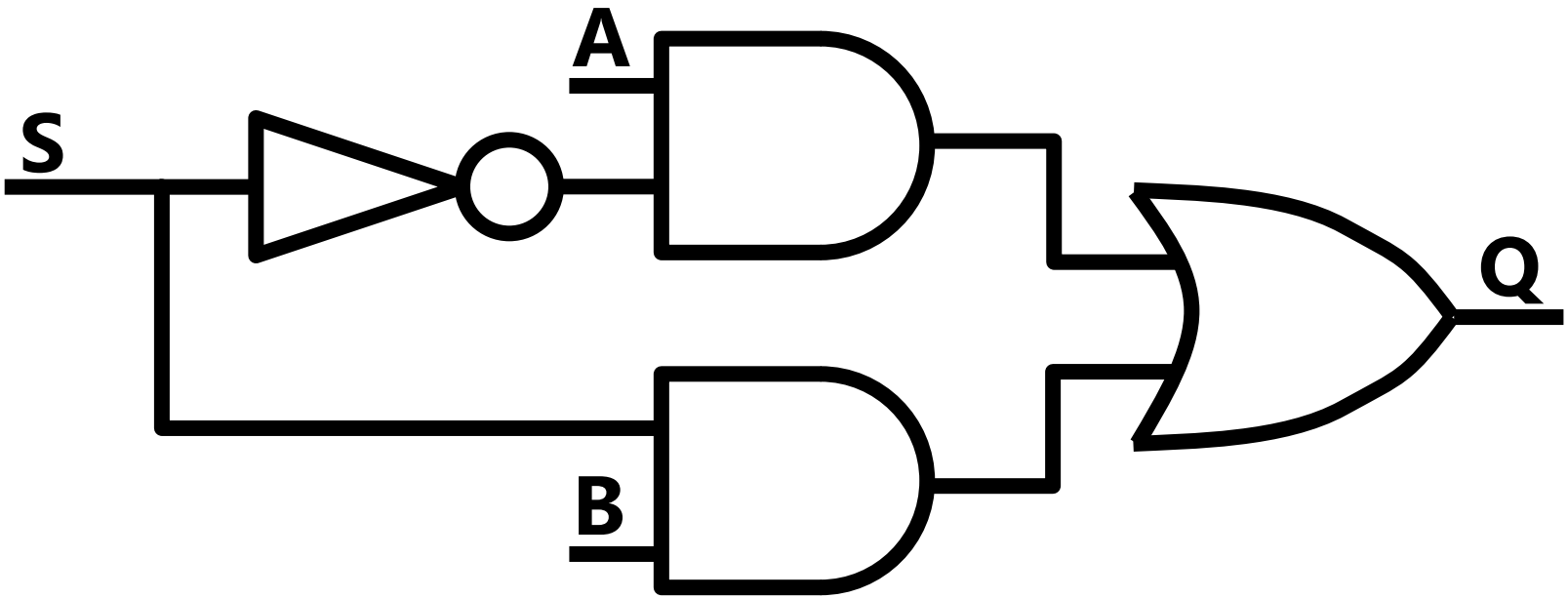
$$\mathbf{Q = \bar{S}A + SB}$$

- Getting the **sum-of-products**
  - an OR of multiple ANDed terms

	S	A	B	Q
	0	0	X	0
	0	0	X	0
$\bar{S}A$	0	1	X	1
~~$\bar{S}A$~~	0	1	X	1
	1	X	0	0
SB	1	X	1	1
	1	X	0	0
~~SB~~	1	X	1	1

Turning that expression into gates

- making $\mathbf{Q = \bar{S}A + SB}$ into gates is pretty straightforward:



Minimization using Boolean Algebra

Boolean algebra

- Idempotent
  - $a.a = a + a = a$
- Commutative
  - $a.b = b.a$
  - $a + b = b + a$
- Associative
  - $a.(b.c) = (a.b).c$
  - $a + (b + c) = (a + b) + c$
- Distributive
  - $a.(b + c) = a.b + a.c$
  - $a + (b.c) = (a + b).(a + c)$

- De Morgan's laws
  - $\overline{a.b} = \bar{a} + \bar{b}$
  - $\overline{a + b} = \bar{a}.\bar{b}$

- Other
  - $a + (a.b) = a$
  - $a.(a + b) = a$
  - $\bar{\bar{a}} = a$
  - $a + \bar{a} = 1$
  - $a.\bar{a} = 0$

What if the function is more complex?

- if we use that method on the $\mathbf{C_{out}}$ of a full adder:

A	B	C_{in}	C_{out}	
0	0	0	0	
0	0	1	0	
0	1	0	0	
0	1	1	1	$\overline{\mathbf{A}}\mathbf{B}\mathbf{C_{in}}$
1	0	0	0	
1	0	1	1	$\mathbf{A}\overline{\boldsymbol{B}}\mathbf{C_{in}}$
1	1	0	1	$\mathbf{A}\mathbf{B}\overline{\mathbf{C_{in}}}$
1	1	1	1	$\mathbf{A}\mathbf{B}\mathbf{C_{in}}$

$$\mathbf{C_{out}} = \overline{\mathbf{A}}\mathbf{B}\mathbf{C_{in}} + \mathbf{A}\overline{\boldsymbol{B}}\mathbf{C_{in}} + \mathbf{A}\mathbf{B}\overline{\mathbf{C_{in}}} + \mathbf{A}\mathbf{B}\mathbf{C_{in}}$$

- It feels too complex, somehow
- each NOT gate uses 2 transistors
- each AND/OR gate uses 6
- this will use **72 transistors**
  - *just* for the carry
  - of *one* one-bit addition

Minimizing equations –WHYYYYY!!

$$\boldsymbol{AB\overline{C} + A\overline{B}C + \overline{A}BC + ABC}$$

72 transistors

Using distributivity $AB + AC = A(B + C)$:

$$AB\left(\overline{C} + C\right) + A\overline{B}C + \overline{A}BC$$

Adjacency #1 tackled:

$$AB + A\overline{B}C + \overline{A}BC$$

What about now? Two variables change ☹

Minimizing equations –WHYYYYY!!

$$\boldsymbol{AB\overline{C} + A\overline{B}C + \overline{A}BC + ABC}$$

Adding (ORing) the same term multiple times is ok:

$$AB + A\overline{B}C + ABC + \overline{A}BC + ABC$$

Note the two adjacencies :D

$$AB + AC\left(\overline{B} + B\right) + BC\left(\overline{A} + A\right)$$

Adjacency #2 and #3 tackled:

$$\boldsymbol{AB + CA + CB}$$



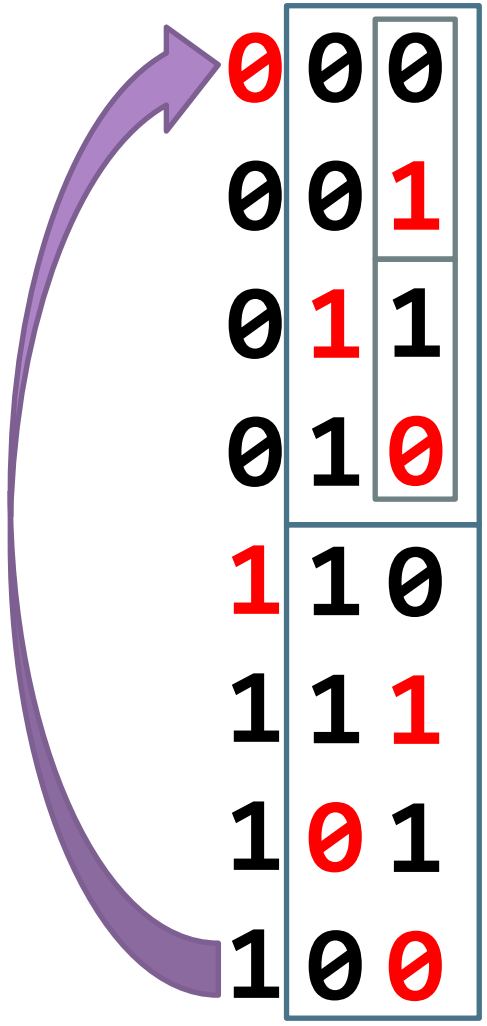
It’s hard!

If only we had a better tool to help us

Karnaugh Maps

Gray Code

- **Gray code** is a way of encoding binary where only **one bit** changes on each step
- How can we construct it?
- Let's revisit our binary number table:
  - Start by creating the first two 1-bit entries
  - Mirror them and complete the next column
  - Repeat!
- See how only one bit changes from the last to the first number!
- We'll only be using 2-bit code.



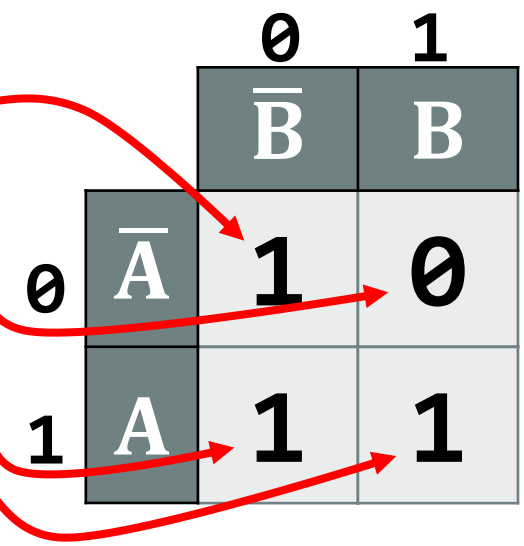
Karnaugh Maps (K-maps) – Setting up

- **Karnaugh Maps** are a tool for minimizing boolean functions
- It helps us finding adjacencies
  - let's start with a function that has two inputs

Truth Table

A	B	Q
0	0	1
0	1	0
1	0	1
1	1	1

K-map



1. Karnaugh maps are represented as a table
2. write input values ***in Gray code*** along axes.
   - (there's only one input on each side here, it's easy)
3. Fill in cells from truth table.

Karnaugh Maps (K-maps) – Finding rects

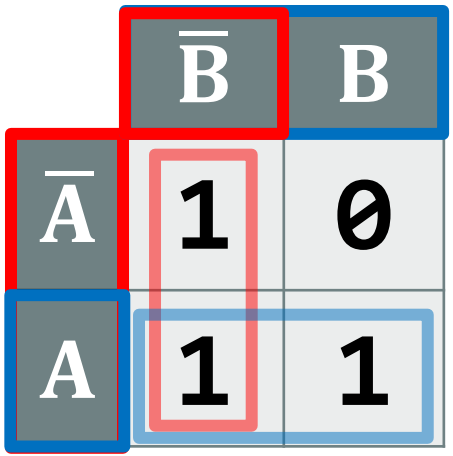
K-map

	$\overline{B}$	B
$\overline{A}$	1	0
A	1	1

3. find **rectangles of 1s** with these rules:
   - width and height can only be 1, 2, or 4
     - NEVER 3
   - overlapping is totally fine! it's *good!*
   - use **the biggest rectangles possible**
   - use **the fewest rectangles possible**

Karnaugh Maps (K-maps) – Interpreting rects

K-map



Red: $\overline{B}$ **Blue: A**

$$Q = A + \overline{B}$$

4. for each rectangle, look at the values of the variables along the axes. some variables **change**, and others don't.
   - which variable changes in the red rectangle? which doesn't?
   - what about the blue rectangle?
5. each rectangle is an AND **minterm**
   - write the variables **that stay the same for that rect** (keeping the NOT bars)
   - ignore the variables that **change**
6. OR all the terms together
7. WHEW!

I'd like to place an order for the carry-out bit

- With more than 2 variables, put two along one axis (**GRAY CODE!**)

C	A	B	C_o
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

try to make the rectangles **as big as possible.** overlap is goooood.

		00	01	11	10
		$\bar{A}\bar{B}$	$\bar{A}B$	AB	$A\bar{B}$
0	$\bar{C}$	0	0	1	0
1	C	0	1	1	1

Red: AC
Green: AB
Blue: BC

$$C_o = AC + AB + BC$$

I'd like to place an order for the carry-out bit

- With more than 2 variables, put two along one axis (**GRAY CODE!**)

C	A	B	C_o
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

try to make the rectangles **as big as possible.** overlap is goooood.

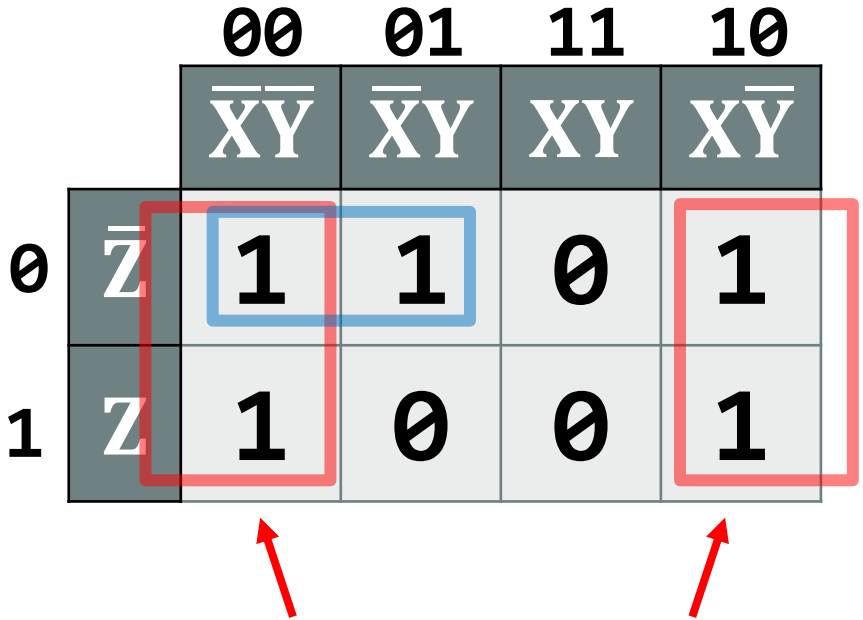
		00	01	11	10
		$\bar{A}\bar{B}$	$\bar{A}B$	AB	$A\bar{B}$
0	$\bar{C}$	0	0	1	0
1	C	0	1	1	1

Did you notice this? ABC

This is the term we needed to add multiple times :D

Just like a 2D RPG world map…

- rectangles on K-maps can *wrap around* (left-right AND top-bottom!)



this is really a **2x2 rectangle.**
it's just… doing its best.

Red: $\bar{Y}$

Blue: $\bar{X}\bar{Z}$

$$Q = \bar{Y} + \bar{X}\bar{Z}$$

Okay, maybe it's not perfect.

- let's try the Sum output of a full adder

a **1x1 rectangle** becomes a term that uses all the variables

		00	01	11	10
		$\bar{A}\bar{B}$	$\bar{A}B$	AB	$A\bar{B}$
0	$\bar{C}_i$	0	1	0	1
1	C_i	1	0	1	0

Red: $\bar{A}\bar{B}C$

Green: $\bar{A}B\bar{C}$

Blue: ABC_i

Purple: $A\bar{B}\bar{C}$

$$\mathbf{Sum} = \bar{A}\bar{B}C + \bar{A}B\bar{C} + ABC + A\bar{B}\bar{C}$$

wait, can't we do that as:

$$\mathbf{Sum} = A \oplus B \oplus C$$ *(that's xor!)*

Tradeoffs, tradeoffs

- there are extensions to K-maps to detect XORs
- but…
  - XOR gates are slower than AND/OR gates
  - if area is a concern, an XOR make sense
  - if speed is a concern, AND/OR gates make sense
- what do real hardware designers do?
  - they use programs to do this stuff for them lol
  - things like GALs, CPLDs, and FPGAs are *reconfigurable hardware* which usually use "sum-of-products" to do logic, so ANDs and ORs are all you've got

Some more examples

- Can you solve this?

		00	01	11	10
		$\overline{A}\overline{B}$	$\overline{A}B$	AB	$A\overline{B}$
00	$\overline{C}\overline{D}$	0	0	0	0
01	$\overline{C}D$	0	0	0	0
11	CD	0	1	1	0
10	$C\overline{D}$	0	1	1	0

- In AB: A is both {0,1}
- In CD: D is both {0,1}

- Eliminate both A,D
- We get

$$CB$$

Some more examples

- Can you solve this?

		00	01	11	10
		$\overline{A}\overline{B}$	$\overline{A}B$	AB	$A\overline{B}$
00	$\overline{C}\overline{D}$	0	0	0	0
01	$\overline{C}D$	1	1	1	1
11	CD	1	1	1	1
10	$C\overline{D}$	0	0	0	0

- In AB: Both have {0,1}
- In CD: C is both {0,1}

- Eliminate both A, B, C
- We get

$$D$$

Some more examples

- Can you solve this?

	00	01	11	10
	$\overline{A}\,\overline{B}$	$\overline{A}B$	AB	$A\overline{B}$
00	$\overline{C}\,\overline{D}$	1	0	0	1
01	$\overline{C}D$	0	0	0	0
11	CD	0	0	0	0
10	$C\overline{D}$	1	0	0	1

- In AB: A has both {0,1}
- In CD: C is both {0,1}

- Eliminate both A, C
- We get

$$\overline{B}\,\overline{D}$$

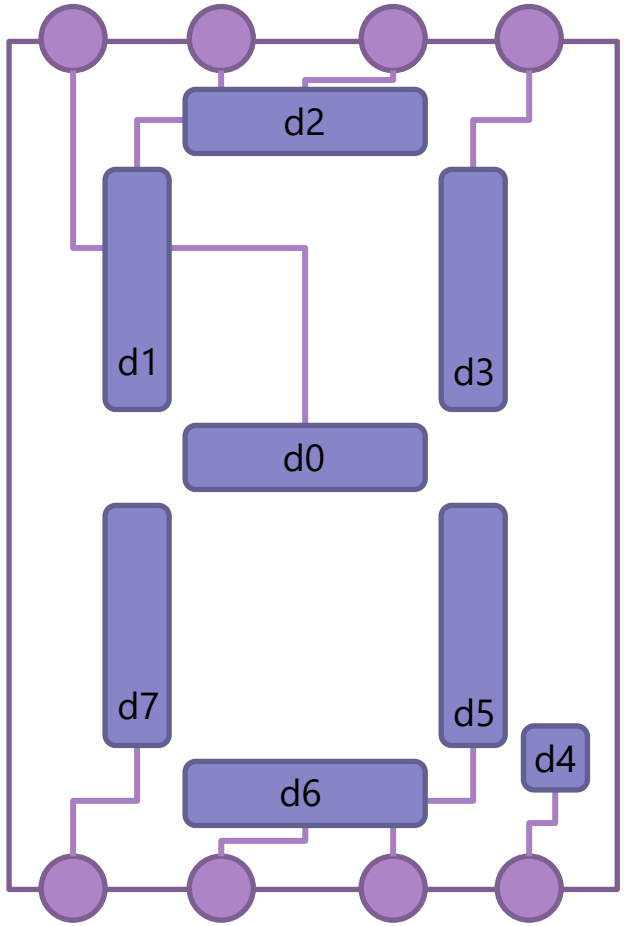
this is still a **2x2 rectangle!**

7 segment LED display

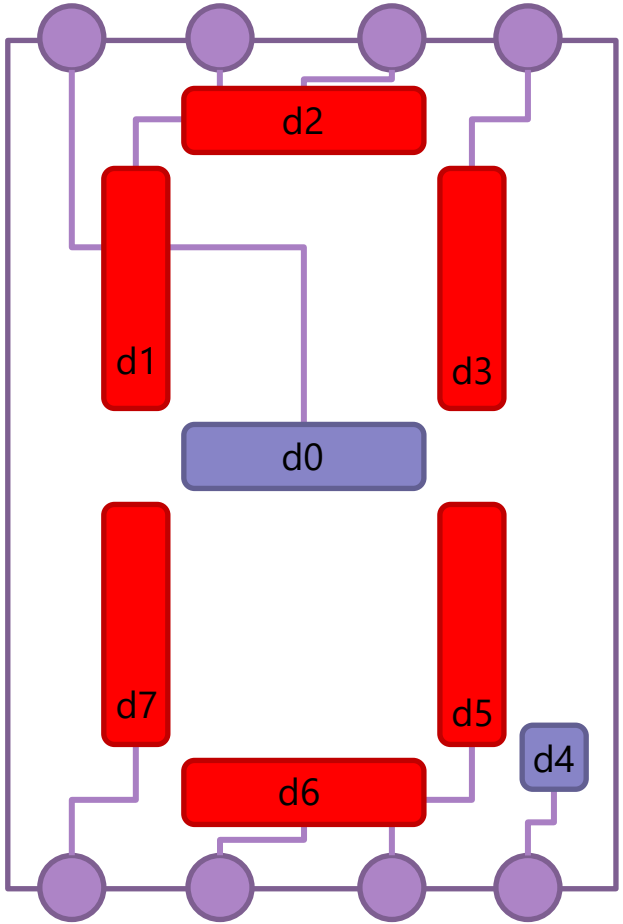
- **This is a 7 segment LED display**
  - It displays numbers
  - It has 8 LEDs (one for the decimal point)

- **Problem**
  - Given a 4-bit number, draw the corresponding numeral.
  - E.g. 0000 is “0”; 1001 is “9”.
  - Ignore the dot

- **Solution**
  - Create a Boolean function for each segment.



7 segment LED display

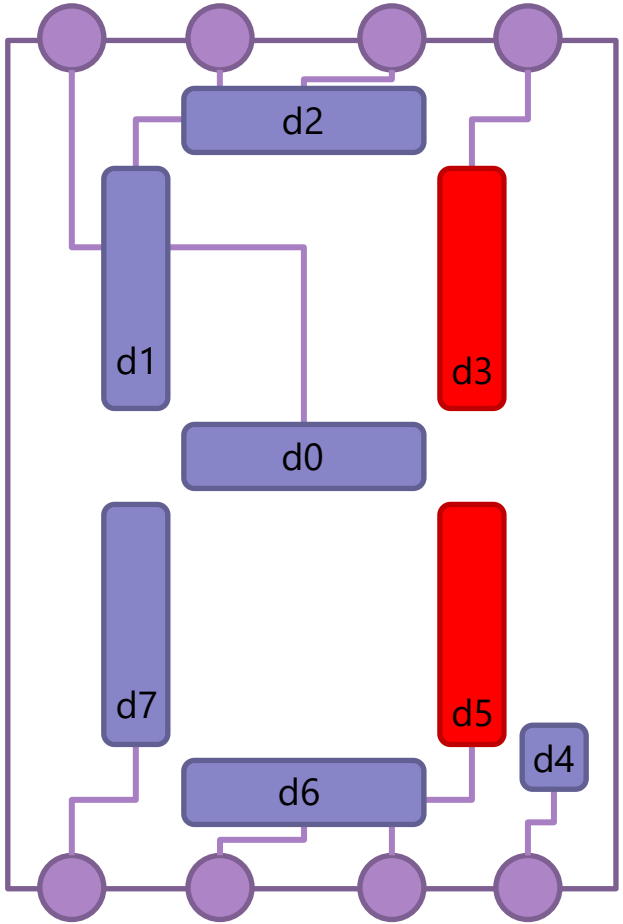


Number 0
01110111

d0 d7

Hex Digit LED
7 segments, 1 decimal point
Turn each segment on/off
State: 0=OFF, 1=ON
"Draw" numbers 0 to 9

7 segment LED display

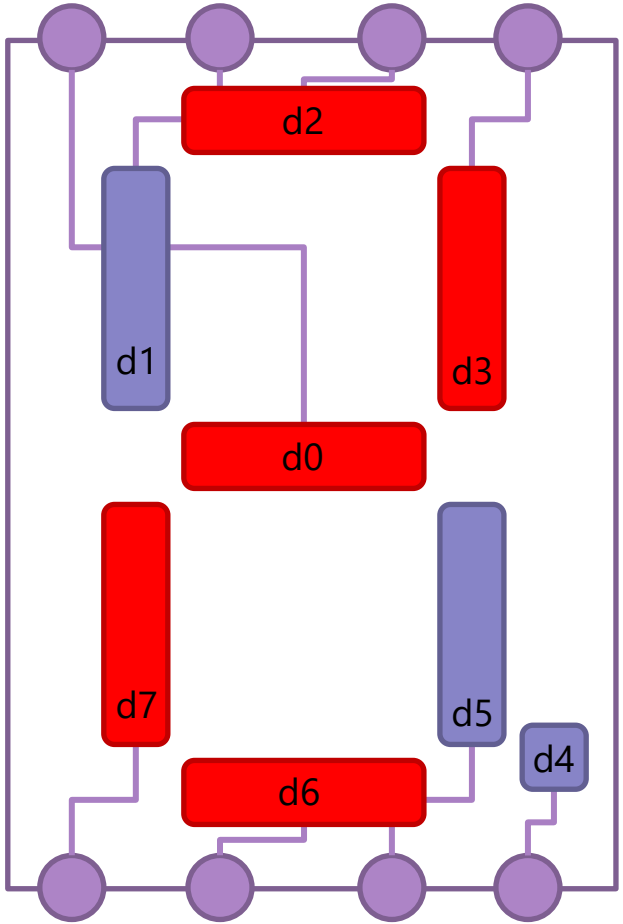


Number 1
00010100

d0 d7

Hex Digit LED
7 segments, 1 decimal point
Turn each segment on/off
State: 0=OFF, 1=ON
"Draw" numbers 0 to 9

7 segment LED display

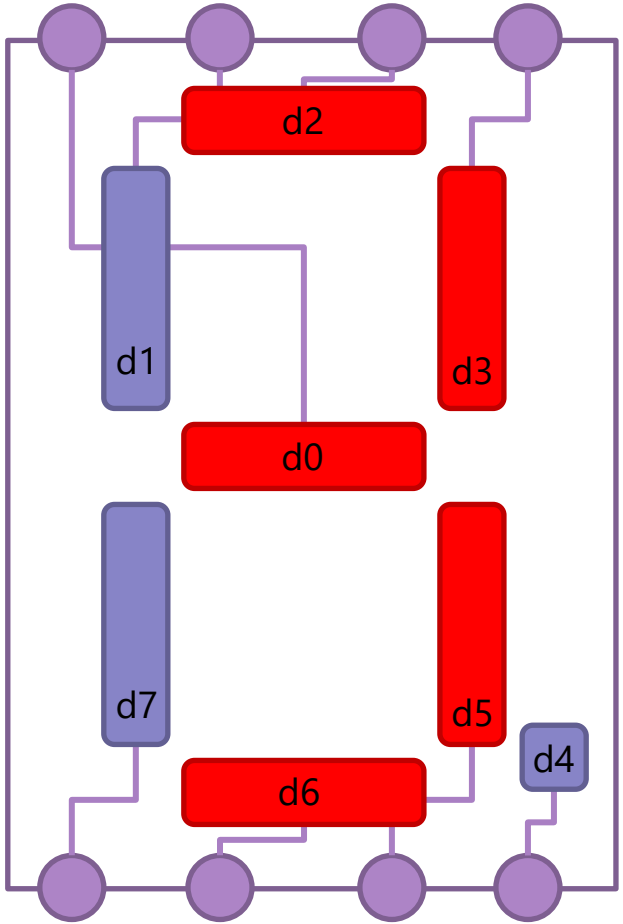


Number 2
10110011

d0 d7

Hex Digit LED
7 segments, 1 decimal point
Turn each segment on/off
State: 0=OFF, 1=ON
"Draw" numbers 0 to 9

7 segment LED display

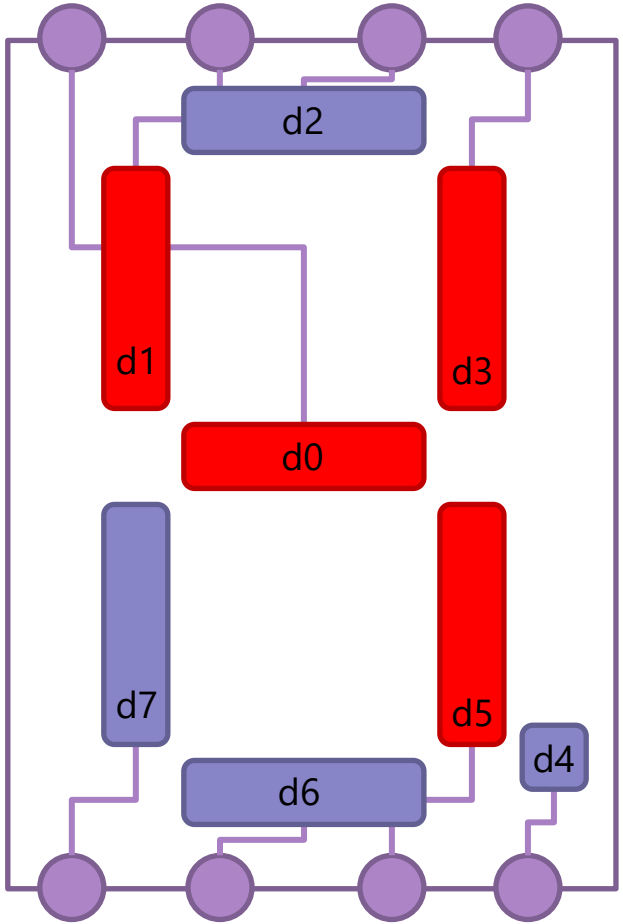


Number 3
10110110

d0 d7

Hex Digit LED
7 segments, 1 decimal point
Turn each segment on/off
State: 0=OFF, 1=ON
"Draw" numbers 0 to 9

7 segment LED display

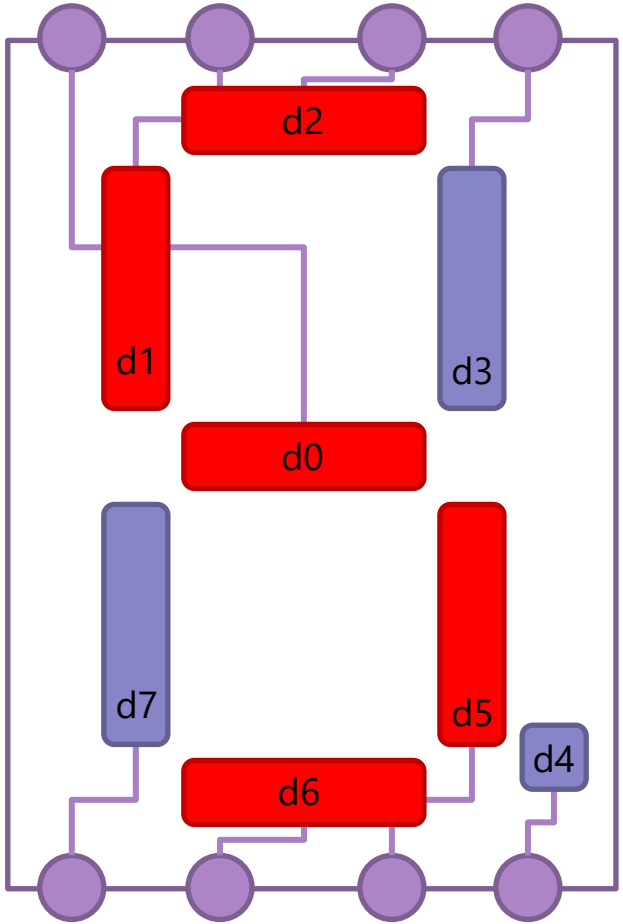


Number 4
11010100

d0 d7

Hex Digit LED
7 segments, 1 decimal point
Turn each segment on/off
State: 0=OFF, 1=ON
"Draw" numbers 0 to 9

7 segment LED display

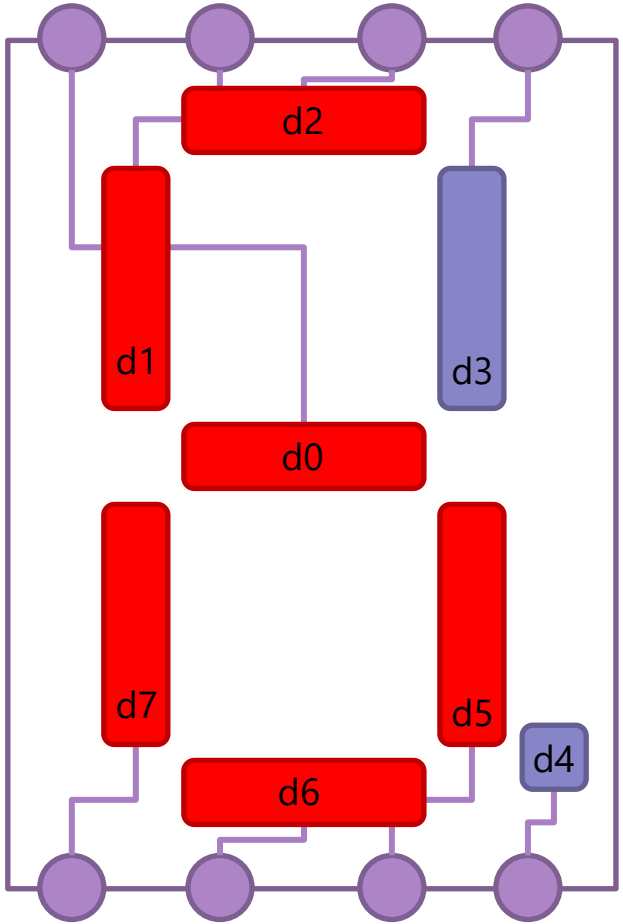


Number 5
11100110

d0 d7

Hex Digit LED
7 segments, 1 decimal point
Turn each segment on/off
State: 0=OFF, 1=ON
"Draw" numbers 0 to 9

7 segment LED display

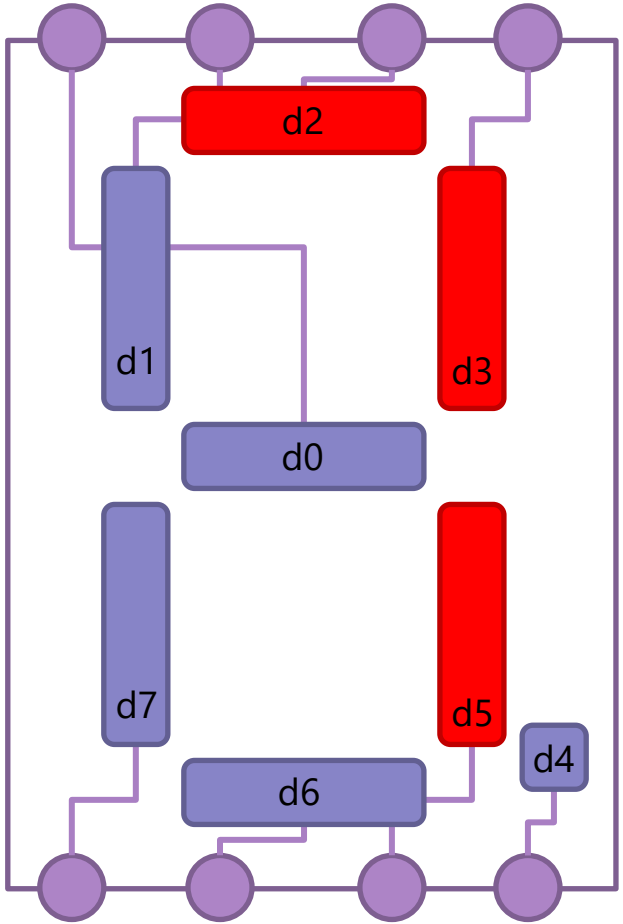


Number 6
11100111

d0 d7

Hex Digit LED
7 segments, 1 decimal point
Turn each segment on/off
State: 0=OFF, 1=ON
"Draw" numbers 0 to 9

7 segment LED display

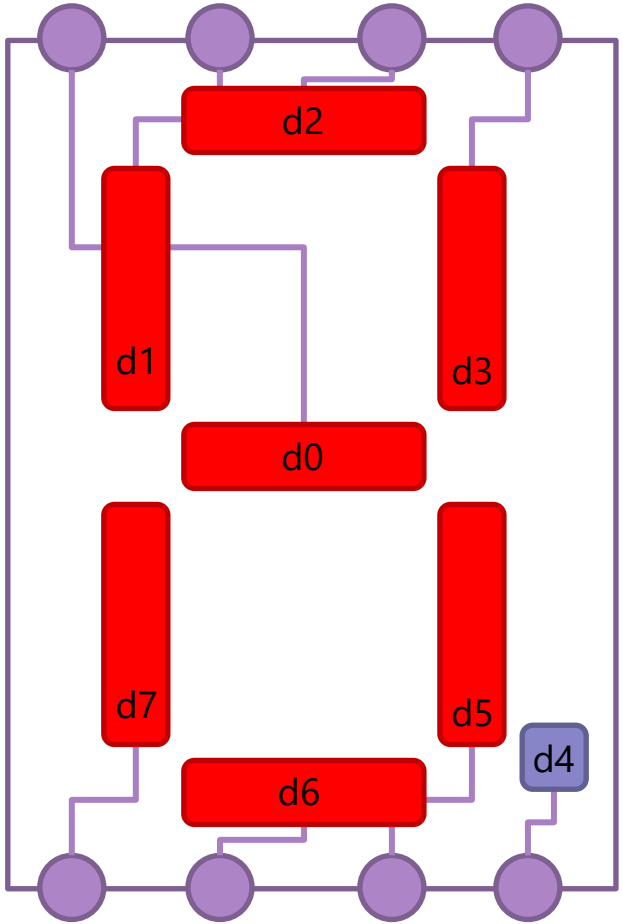


Number 7
00110100

d0 d7

Hex Digit LED
7 segments, 1 decimal point
Turn each segment on/off
State: 0=OFF, 1=ON
"Draw" numbers 0 to 9

7 segment LED display

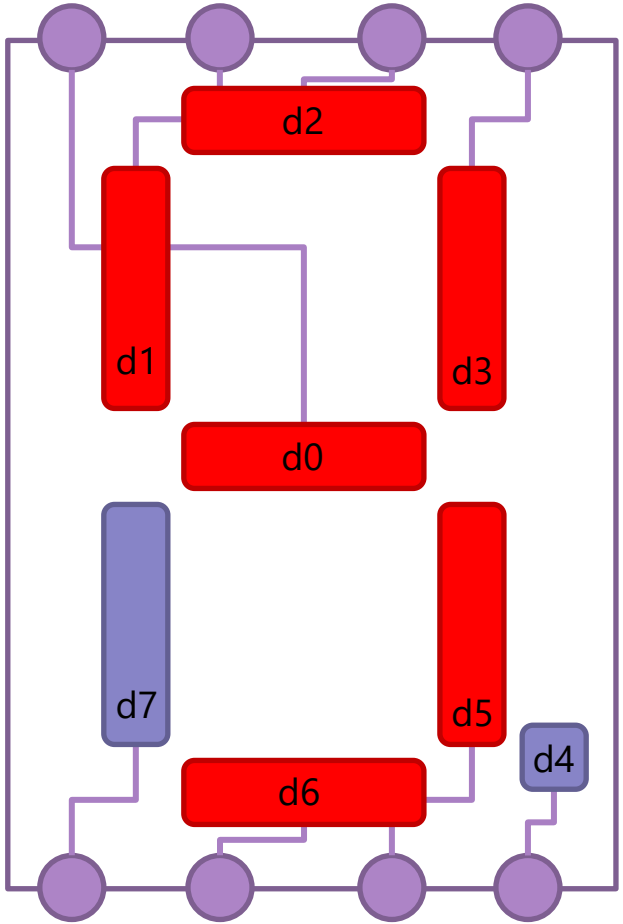


Number 8
11110111

d0 d7

Hex Digit LED
7 segments, 1 decimal point
Turn each segment on/off
State: 0=OFF, 1=ON
"Draw" numbers 0 to 9

7 segment LED display



Number 9
11110110

d0 d7

Hex Digit LED
7 segments, 1 decimal point
Turn each segment on/off
State: 0=OFF, 1=ON
"Draw" numbers 0 to 9

7 segment LED display

How to approach this?

- Create a truth table
  - Inputs are i0 to i3 (4 bits)
  - Outputs are numbered d0, to d7, corresponding to segments

- Minimize the circuit using a K-map
  - Create the table
  - Follow the rules!
- Draw the numerals by setting d0 to d7 to 1s or 0s
  - Build the circuit!

7 segment LED display

i3	i2	i1	i0	d0	d1	d2	d3	d4	d5	d6	d7
0	0	0	0	0	1	1	1	0	1	1	1
0	0	0	1	0	0	0	1	0	1	0	0
0	0	1	0	1	0	1	1	0	0	1	1
0	0	1	1	1	…	…	…	…	…	…	…
0	1	0	0	1	…	…	…	…	…	…	…
0	1	0	1	1	…	…	…	…	…	…	…
0	1	1	0	1	…	…	…	…	…	…	…
0	1	1	1	0	…	…	…	…	…	…	…
1	0	0	0	1	…	…	…	…	…	…	…
1	0	0	1	1	…	…	…	…	…	…	…
1	0	1	0	X	X	X	X	X	X	X	X
…	…	…	…	…	…	…	…	…	…	…	…

7 segment LED display

i3	i2	i1	i0	d0	d1	d2	d3	d4	d5	d6	d7
0	0	0	0	0	1	1	1	0	1	1	1
0	0	0	1	0			1	0	1	0	0
0	0	1					1	0	0	1	
0	0	1					…	…			
0	1	0					…	…			
0	1	0					…				
0	1	1					…				
0	1	1					…				
1	0	0	0	1		…	…				
1	0	0	1	1	…	…	…	…			…
1	0	1	0	X	X	X	X	X	X	X	X
…	…	…	…	…	…	…	…	…	…	…	…

It can only display numbers up to 9!

So we "don't care" about the outputs for larger numbers

7 segment LED display

- Now use a K-map for each output function d0-d7 - Let's start with d0

		00	01	11	10
		$\overline{i_1}\,\overline{i_0}$	$\overline{i_1} i_0$	$i_1 i_0$	$i_1\,\overline{i_0}$
00	$\overline{i_3}\,\overline{i_2}$	0	0	1	1
01	$\overline{i_3} i_2$	1	1	0	1
11	$i_3 i_2$	X	X	X	X
10	$i_3\,\overline{i_2}$	1	1	X	X

i3	i2	i1	i0	d0
0	0	0	0	0
0	0	0	1	0
0	0	1	0	1
0	0	1	1	1
0	1	0	0	1
0	1	0	1	1
0	1	1	0	1
0	1	1	1	0
1	0	0	0	1
1	0	0	1	1
1	0	1	0	X
…	…	…	…	…

7 segment LED display

- Now use a K-map for each output function d0-d7
- Let's start with d0

		00	01	11	10
		$\overline{i_1}\,\overline{i_0}$	$\overline{i_1} i_0$	$i_1 i_0$	$i_1\,\overline{i_0}$
00	$\overline{i_3}\,\overline{i_2}$	0	0	1	1
01	$\overline{i_3} i_2$	1	1	0	1
11	$i_3 i_2$	X	X	X	X
10	$i_3\,\overline{i_2}$	1	1	X	X

Output "don't cares" can be whatever we need them to be ☺

7 segment LED display

	00	01	11	10
	$\overline{i_1}\,\overline{i_0}$	$\overline{i_1}\,i_0$	$i_1 i_0$	$i_1\,\overline{i_0}$
00 $\overline{i_3}\,\overline{i_2}$	0	0	1	1
01 $\overline{i_3}\,i_2$	1	1	0	1
11 $i_3 i_2$	X	X	X	X
10 $i_3\,\overline{i_2}$	1	1	X	X

- Four minterms:
  - i_3
  - $i_2\overline{i_1}$
  - $i_1\overline{i_2}$
  - $i_1\overline{i_0}$

$$d_0 = i_3 + i_2\overline{i_1} + i_1\overline{i_2} + i_1\overline{i_0}$$

On your own

- Try making a circuit in Logisim using the 7 segment LED display
  - Find it in input/output
- Connect the logic you just solved into d0 (top-left pin) and test it
- Solve for the remaining segments
  - It's fun and practice